GEPA in depth

Intent

GEPA is DSPy's reflection-driven instruction optimizer. It maintains a population of candidate programs, scores each on a validation set, and uses an LLM "reflection" step to propose instruction edits informed by per-predictor natural-language feedback from your metric. The Pareto-frontier sampling and reflective proposal mechanics are what distinguish it from COPRO and MIPROv2; the metric contract — `dspy.Prediction(score, feedback)` — is what makes the feedback channel work at all.

Read this when the selection guide has pointed you at GEPA and you want the full mechanics: how the budget translates into evaluations, how feedback reaches the reflection LM, what `auto` modes actually buy, and what `detailed_results` carries when you track stats.

Design decisions

1. The metric is the feedback channel — return `Prediction(score, feedback)`, not a bare float

GEPA reads the metric's `feedback` field directly into the reflection prompt. A plain-float metric still works, but the proposer sees only a generic "This trajectory got a score of {n}" caption — concrete failure modes never reach it. GEPA doesn't error on a float; it just gives you a much weaker version of itself.

2. Reflection is the proposal mechanism, not the evaluation mechanism

`reflection_lm` is called once per mutation to read score-logging traces and emit a new instruction. Score evaluation runs through whatever LM the program is configured with (`task_model`). Two roles, two budgets: reflection is rare and benefits from a strong LM; evaluation is frequent and benefits from a cheap one. Passing `reflection_lm=dspy.LM("openai/gpt-4o")` while running the program on `gpt-4o-mini` is the standard configuration.

3. Only one of `auto`, `max_full_evals`, or `max_metric_calls` may be set

GEPA enforces this at construction. The three knobs are different units of the same budget: `auto` is a preset (light / medium / heavy), `max_full_evals` is "this many passes over trainset + valset," `max_metric_calls` is the raw ceiling on metric invocations. Pick one shape and let GEPA derive the others.

4. The Pareto frontier is the search space; the aggregate score is the winner

GEPA tracks every candidate it ever proposed, with per-example scores. When picking the next program to mutate, it samples stochastically from the per-example Pareto frontier — programs that score best on at least one example. When returning a winner from `.compile()`, it picks the single program with the highest aggregate score. The frontier is for exploration; the aggregate is for selection.

5. Per-predictor feedback comes through `pred_name` and `pred_trace`

For each predictor GEPA wants to update, it calls the metric twice: once at module level (`pred_name=None, pred_trace=None`) for the aggregate score, and once with `pred_name="my_predictor"` and `pred_trace=[(predictor, inputs, outputs)]` (the subtrace of just that predictor's call). The metric can grade that step in isolation; most users return the same module-level score for both, and that's fine. `warn_on_score_mismatch=True` (default) logs when the two diverge.

6. `skip_perfect_score=True` keeps reflection focused on what's broken

Examples scoring at `perfect_score` (default `1.0`) drop out of the reflective minibatch. The reflection LM then sees only failures, which is what you want — proposals informed by perfect runs would suggest changes to working behavior. Lower `perfect_score` if your metric saturates below `1.0`.

7. `use_merge=True` combines successful candidates

When two candidates each win on different examples, merging proposes a new candidate that inherits instructions from both. Merging costs one re-evaluation per attempt; `max_merge_invocations` (default 5) caps the total. Set it to `None` to disable. Worth keeping on for most runs — merges find combinations the proposer wouldn't.

8. The default `component_selector` is round-robin across predictors

On each iteration, GEPA picks one predictor to update. Round-robin cycles through them. `"all"` updates every predictor at once — saves iterations when predictors are tightly coupled. A custom `ReflectionComponentSelector` lets you target hot predictors. Round-robin is the safe default.

9. Threading is per-batch, not per-mutation

`num_threads` parallelizes the metric calls inside each evaluation pass; the mutation loop itself is serial. Scaling threads helps when the valset is large and the metric is slow, not when the reflection step is the bottleneck. If `reflection_lm` is your slow path, more threads do nothing.

10. `detailed_results` is the audit trail

When `track_stats=True`, the compiled program's `detailed_results` carries every candidate, every parent in the lineage, every per-example score, and when each candidate was discovered. Use it to understand whether GEPA exhausted its budget productively or burned cycles on near-duplicates.

11. The winner's instructions overwrite the student's signatures

`.compile()` returns a Module with the best candidate's instructions baked into each predictor's `signature.instructions`. Nothing else moves — demos, callbacks, and sub-modules stay as the student had them. `program.save(path)` writes the instructions; loading on a different machine reapplies them.

12. Reflection LM cost dominates if you're not careful

A `medium` budget on a 2-predictor / 100-example task means ~12 mutations, each spawning 1–3 reflection calls depending on minibatch size and number of components — so ~12–36 calls to `reflection_lm`. With `gpt-4o` reflecting and `gpt-4o-mini` running the program, reflection often costs more than evaluation. Budget accordingly.

API walkthrough

Grouped by what you're trying to do.

The optimizer

```
dspy.GEPA(metric, *, auto=None, max_full_evals=None, max_metric_calls=None,
reflection_minibatch_size=3, candidate_selection_strategy="pareto",
reflection_lm=None, skip_perfect_score=True,
add_format_failure_as_feedback=False, instruction_proposer=None,
component_selector="round_robin", use_merge=True, max_merge_invocations=5,
num_threads=None, failure_score=0.0, perfect_score=1.0, log_dir=None,
track_stats=False, use_wandb=False, use_mlflow=False, seed=0, ...)
```

Takes a feedback-shaped metric, a budget (exactly one of `auto` / `max_full_evals` / `max_metric_calls`), and a `reflection_lm` (defaults to the globally-configured LM — you'll often want to pass a stronger one). `candidate_selection_strategy="current_best"` switches off Pareto sampling and turns GEPA into greedy local search. `failure_score` is the score assigned when the metric raises; `perfect_score` is the threshold for `skip_perfect_score`.

```
GEPA.compile(student, *, trainset, valset=None, ...) -> dspy.Module
```

Runs the search. `valset` defaults to `trainset` when not provided. Returns a fresh Module whose predictors carry the best candidate's instructions; sets `_compiled = True`.

Budget translation

How a budget number turns into LM calls.

`auto` — light / medium / heavy preset Maps to `num_candidates = 6 / 12 / 18` and derives the metric-call budget from your valset size. The conversion accounts for the initial full eval, bootstrapping (`3 * num_candidates`), per-candidate minibatch evals, and periodic full evals every 5 steps. On a 2-predictor / 100-example task: light ≈ 1330 calls, medium ≈ 1740, heavy ≈ 2045.

`max_full_evals` — pass-count budget Each "full eval" is one walk through the union of trainset and valset. GEPA computes `max_metric_calls = max_full_evals * (len(trainset) + len(valset))` internally.

`max_metric_calls` — raw cap Direct ceiling on metric invocations. Use this when you've measured per-call cost and want a hard dollar cap.

The reflective loop

Metric shape — `metric(gold, pred, trace, pred_name, pred_trace) -> dspy.Prediction(score, feedback)` Called twice per evaluated example: once at module level (`pred_name=None`) for the aggregate, once per predictor under mutation (`pred_name="..."`, `pred_trace=[(predictor, inputs, outputs)]`) for per-predictor feedback. Return `dspy.Prediction(score: float, feedback: str)`. The `feedback` string is what reaches the reflection prompt verbatim.

`reflection_lm` — the LM that proposes new instructions Reads a minibatch of low-scoring traces and emits a candidate instruction for the selected predictor. Defaults to the globally configured LM; passing a stronger one explicitly is the most common GEPA tuning. Called serially per mutation — threading doesn't help here.

`reflection_minibatch_size` — examples shown to the reflection LM per mutation Default `3`. Larger minibatches give the proposer more context (better proposals) at the cost of longer reflection prompts (more tokens).

`instruction_proposer` — custom proposer hook A callable matching the `gepa.ProposalFn` protocol: takes a `{predictor_name: current_instruction}` dict, a reflective dataset of low-scoring examples, and a list of components to update; returns a new `{predictor_name: new_instruction}` dict. Override when the default proposer doesn't handle your modality (signatures with `dspy.Image` fields, say) or when you want domain-specific constraints on the instructions.

Population dynamics

`candidate_selection_strategy` — `"pareto"` (default) or `"current_best"` `"pareto"` samples next-mutation candidates stochastically from the per-example Pareto frontier. `"current_best"` always mutates the highest-aggregate candidate. Pareto explores more; current-best converges faster on simple tasks.

`component_selector` — `"round_robin"` (default), `"all"`, or a custom `ReflectionComponentSelector` Which predictor to update on each iteration. Round-robin cycles; `"all"` updates every predictor at once. Custom selectors can target predictors by historical regret or any other policy.

`use_merge` / `max_merge_invocations` Merging combines instructions from two successful candidates into a new candidate. Costs one re-evaluation per merge; `max_merge_invocations` (default 5) caps the total.

`skip_perfect_score` / `perfect_score` Examples at `perfect_score` drop out of the reflective minibatch. Default `perfect_score=1.0`; set lower if your metric saturates below 1.

Inspecting the run

`compiled_program.detailed_results` (when `track_stats=True`) → `DspyGEPAResult`

- `candidates: list[Module]` — every program ever proposed
- `parents: list[list[int] | None]` — lineage (`None` for seed, otherwise parent indices)
- `val_aggregate_scores: list[float]` — one per candidate
- `val_subscores: list[list[float]]` — per-candidate, per-example scores
- `per_val_instance_best_candidates: list[set[int]]` — which candidates won which example
- `discovery_eval_counts: list[int]` — metric calls consumed before each candidate appeared
- `best_idx: int` — index of the returned winner
- `best_candidate: Module` — same module GEPA returned from `.compile()`

Convert to a JSON-safe dict via `.to_dict()`.

`log_dir` — disk-side logging When set, GEPA writes per-iteration logs to that directory: candidates, scores, proposed instructions. Useful for post-mortem on what the reflection LM was suggesting.

`use_wandb` / `use_mlflow` Stream search progress to Weights & Biases or MLflow. Each candidate's aggregate score is logged as an iteration step.

Cross-links

- Optimizers: choosing one — the selection guide that recommends GEPA for feedback-rich tasks.
- Metrics and evaluation — the `Prediction(score, feedback)` shape GEPA requires, and the LLM-as-judge pattern for building one.
- Modules: composing your own — `_compiled` propagation and the deepcopy-then-mutate pattern GEPA uses on the student.
- BootstrapFewShot family — the demo-tuning alternative; pair with GEPA via `BetterTogether` when both knobs need to turn.